Integración por Monte Carlo con OpenMP — Borrador de respuestas

Estado: completo con las mediciones del equipo (Intel Core i5-12450HX). Entregables que acompañan este documento: `montecarlo.c`, `speedup.png`, `cpuinfo.txt`, `bench.sh`, `grafica.py`.

Parte I. Fundamento del método

I.1 ¿Por qué el error disminuye como 1/√N?

El estimador es $I_N = \frac{b-a}{N}\sum_{i=1}^{N} f(x_i)$ como se presenta en el documento, es decir, un promedio de N variables aleatorias independientes y distribuidas de manera idéntica. Si cada término $(b-a)f(x_i)$ tiene varianza de σ^2, la varianza de un promedio de variables independientes es, en realidad, la varianza individual dividida entre N:

$$\mathrm{Var}(I_N) = \frac{\sigma^2}{N} \quad \Rightarrow \quad \text{error típico} = \frac{\sigma}{\sqrt{N}}$$

Cada muestra nueva aporta información, pero los errores positivos y negativos se cancelan parcialmente; esa cancelación crece como $\sqrt{N}$ y no como N.

Para reducir el error a la mitad se necesita $\frac{\sigma}{\sqrt{N'}} = \frac{1}{2}\frac{\sigma}{\sqrt{N}} \Rightarrow N' = 4N$ muestras adicionales . Es decir, hay que cuadruplicar las muestras (agregar $3N$ adicionales).

Reducir el error 10 veces exige $N' = 100N$. Cada dígito de precisión cuesta 100 veces más muestras, un costo exponencial en el número de dígitos. Por eso Monte Carlo es poco eficiente para alta precisión en una dimensión, y es valioso sobre todo cuando la dimensión es grande ya que su tasa $1/\sqrt{N}$ no depende de la dimensión.

I.2 ¿Por qué es embarazosamente paralelo?

Cada muestra (x_i, y_i) se genera y se evalúa sin usar el resultado de ninguna otra muestra, ósea que no hay dependencia de datos entre iteraciones. Lo único que las conecta es el acumulado de aciertos, y la suma es una operación asociativa y conmutativa, por lo que el orden de acumulación no cambia el resultado. Esto provoca que se pueda repartir el trabajo sin comunicación ni sincronización durante el cálculo.

I.3 El reparto entre p hilos produce el mismo estimador

Sea $X_i \in \{0, 1\}$ el indicador de acierto de la muestra i. Se reparten los índices en p conjuntos disjuntos $B_1, \dots, B_p$ cuya unión es $\{1, \dots, N\}$. El conteo del hilo j es $a_j = \sum_{i \in B_j} X_i$ y

$$\sum_{j=1}^{p} a_j = \sum_{j=1}^{p} \sum_{i \in B_j} X_i = \sum_{i=1}^{N} X_i,$$

porque los B_j son una partición y la suma de enteros es asociativa y conmutativa. Entonces $4 \sum_j a_j / N$ es exactamente el estimador secuencial, siempre y cuando que se evalúen las mismas N muestras.

Con el mismo N total, la varianza es σ^2/N sin importar cuántos hilos haya, porque lo que fija la precisión es el número total de muestras independientes y no cuántos hilos las generen. Paralelizar no mejora la precisión estadística, solo reduce el tiempo.

Parte II. Implementación

II.1 Verificación de la estimación (parte analítica)

Para el acierto-fallo, el acierto es una variable Bernoulli con $q = \pi/4$, y $\hat{\pi} = 4\hat{q}$, de modo que

$$\sigma_{\hat{\pi}} = 4\sqrt{\frac{q(1-q)}{N}} = 4\sqrt{\frac{0{,}1686}{10^8}} \approx 1{,}6 \times 10^{-4}.$$

De ahí sale el orden de 10^{-4}, es la desviación estándar teórica del estimador con $N = 10^8$, que se cae directamente de $1/\sqrt{N} = 10^{-4}$ multiplicado por una constante de orden 1.

Mi ejecución con semilla 2225112 (código de estudiante) y $N = 10^8$ dio $\hat{\pi} = 3{,}141542$, con un error absoluto de $5{,}1 \times 10^{-5} \approx 0{,}3\sigma$, lo cual es consistente con lo esperado.

EL Tiempo medido en la versión secuencial (compilada sin `-fopenmp`) tardó $T(1) = 0{,}198$ s en una ejecución, y la versión OpenMP con $p = 1$ tuvo una mediana de $0{,}212$ s (cinco corridas: 0,195 a 0,221 s).

II.2 Decisiones de diseño (resumen para el código `montecarlo.c`)

- **Generador:** sSe uso SplitMix64 con salto directo. El estado k-ésimo es `s_k = SEMILLA + k·GAMMA (mod 2^64)` y el número k-ésimo es una función de mezcla de

`s_k`. Como `s_k` es una fórmula cerrada en k, se puede saltar a cualquier posición sin tener que recorrer la secuencia entera.

- **Flujos sin solapamiento:** la muestra i usa las posiciones $2i$ y $2i+1$ de una sola secuencia maestra de periodo 2^{64}. Cada hilo recorre un tramo disjunto (con `schedule(static)`, un bloque contiguo de $\approx N/p$ muestras), es decir, las semillas de los hilos están espaciadas en $2N/p \geq N/p$ posiciones. Nada se solapa, y no hay que espaciar semillas a mano.
- **Reproducibilidad para cualquier p:** como cada muestra depende solo de su índice y no del hilo, el conteo es idéntico para todo p.
- **Por qué no se uso `rand_r`:** solo tiene 32 bits de estado y no permite saltar posiciones, así que no puede espaciar flujos ni garantizar la reproducibilidad entre distintos p. Además, `rand_r` no existe en MinGW/MSYS2.
- **OpenMP:** `#pragma omp parallel for reduction(+:dentro) schedule(static)`; el número de hilos se elige con el segundo argumento (`omp_set_num_threads`) o con `OMP_NUM_THREADS`.
- **Detalle encontrado:** con `schedule(runtime)` y `OMP_SCHEDULE` sin definir, libgomp usa `dynamic,1` por defecto, lo que hizo la versión 6 veces más lenta (2,06 s vs 0,34 s con 1 hilo). Por eso la planificación quedó fija en `static`, con una macro `PLAN` para recompilar con `dynamic` en II.5.

II.3 ¿Por qué no compartir `rand()` entre hilos?

Problema 1: condición de carrera sobre el estado interno. `rand()` lee el estado global de la variable, calcula el siguiente estado y lo escribe de vuelta en esta. Si dos hilos hacen esto a la vez sin protección, pueden leer el mismo estado, devolver el mismo número y sobrescribirse mutuamente. Como consecuencias se tienen 3: (i) resultados no reproducibles, ya que cambian en cada ejecución; (ii) números repetidos entre hilos, lo que reduce las muestras efectivas por debajo de N; (iii) un estado dañado puede producir secuencias degeneradas o ciclos cortos que sesgan la estimación.

Problema 2: correlación por semilla idéntica. Si cada hilo inicializa su generador con la misma semilla (por ejemplo en MSVCRT/Windows, donde `rand()` tiene estado por hilo y cada hilo nuevo arranca igual), los p hilos generan exactamente la misma secuencia, es decir, los mismos puntos repetidos p veces. Entonces N muestras equivalen a solo N/p puntos distintos.

Efecto sobre $\hat{\pi}$ y su error reportado: cada punto sigue siendo uniforme, de modo que $\hat{\pi}$ sigue siendo en esperanza insesgado, pero su varianza real es la de N/p muestras, es decir, el error real es $\sqrt{p}$ veces mayor que el teórico $\sigma/\sqrt{N}$ que es el programa asume. Con $p = 8$ el error real sería mas o menos $\approx 2{,}8$ veces mayor, mientras que el error reportado sería engañosamente optimista. En cambio, la carrera de datos del problema 1 sí puede introducir sesgo y falta de reproducibilidad.

II.4 Atributos de compartición de datos

Variable	Atributo	Descripción
`N`	compartida (por defecto)	solo lectura dentro de la región
`dentro`	**reducción** (copia privada por hilo)	acumulador; ver abajo
`argc`, `argv`, `p`	compartidas	no se usan dentro del ciclo
`i`	**privada**	la variable del ciclo `for` paralelo es privada automáticamente
`x`, `y`	**privadas**	declaradas dentro del ciclo, cada hilo tiene las suyas
`SEMILLA`, `GAMMA`	no son variables	constantes de preprocesador

Como cláusula se uso `reduction(+:dentro)`. OpenMP crea una copia privada de `dentro` en cada hilo, inicializada en 0 (el elemento neutro de `+`), cada hilo acumula sus aciertos sin interferir con otros y, al terminar, las copias se combinan con `+` sobre la variable original.

Sin la cláusula, `dentro` sería compartida y `dentro++` (leer, sumar, escribir) no seria atómico. Es una condición de carrera. Como consecuencia el conteo queda menor que el real, $4 \cdot \text{dentro}/N$ subestima π de forma sistemática, y el resultado cambia entre ejecuciones (es decir, se vuelve no determinista).

Como evidencia diagnóstica, se puede ejecutar varias veces con el mismo $p > 1$ y observar `aciertos` distintos y siempre menores que el de $p = 1$. Además, todos los hilos escribiendo la misma línea de caché generan false/true sharing y el programa puede volverse más lento que el secuencial.

En la evidencia experimental (variante de codigo `montecarlo_carrera.c`, idéntica pero sin la cláusula `reduction`): se obtuvieron 19 631 692 y 19 637 011 aciertos, frente a 78 538 546 correctos, es decir, $\hat{\pi} \approx 0{,}785$ en lugar de $3{,}1415$ (aproximadamente un cuarto del valor real, y distinto entre corridas), y además tardó 0,58 s frente a 0,34 s de la versión correcta. Con `-O2`, cada hilo acumula en un registro y al final escribe su valor sobre la misma variable compartida, de modo que sobrevive el parcial de un solo hilo.

II.5 (Opcional) `static` vs `dynamic`

Mediciones con $N = 10^8$ y $p = 8$ (mediana de 5 corridas):

Planificación	Mediana [s]
`schedule(static)`	0,053
`schedule(dynamic,10000)`	0,051

La diferencia es de 2 ms, mas o menos un 4 %. Esta es del orden de la resolución del reloj y de la dispersión entre corridas, por lo que no hay un efecto observable de la política de planificación. Y esto en realidad es lo esperado, ya que todas las iteraciones cuestan prácticamente lo mismo, así que el reparto estático en bloques iguales ya está balanceado y `dynamic` no tiene desbalance que corregir; su costo extra es despreciable frente al trabajo. Una posible ventaja de `dynamic` en este procesador híbrido sería compensar hilos más lentos (núcleos E), y el 4 % observado apunta en esa dirección, pero la medición no es concluyente.

Parte III. Desempeño

III.1 Ley de Amdahl: $S(p) = \dfrac{1}{(1-f) + f/p}$

Caso	Cálculo	S	$E = S/p$
(a) $f = 0{,}98$, $p = 8$	$1/(0{,}02 + 0{,}1225)$	**7,02**	87,7 %
(b) $f = 0{,}90$, $p = 8$	$1/(0{,}10 + 0{,}1125)$	**4,71**	58,8 %
(c) $f = 0{,}98$, $p \to \infty$	$1/(1-f) = 1/0{,}02$	**50**	→ 0 %

- a) con solo 2 % secuencial se logra casi el ideal (7,02 de 8).
- b) pasando de 2 % a 10 % de parte secuencial, el speedup cae de 7,02 a 4,71 y se desperdician más de 40 % de los recursos.
- c) aun con infinitos procesadores, el speedup nunca supera 50.

El caso (c) nos enseña que la fracción secuencial $(1-f)$ impone un techo $1/(1-f)$ que ninguna cantidad de hardware puede superar; los últimos porcentajes secuenciales de un programa son los que más pesan. La eficiencia decae al sumar procesadores, así que optimizar la parte secuencial importa tanto como paralelizar.

Equipo de medición

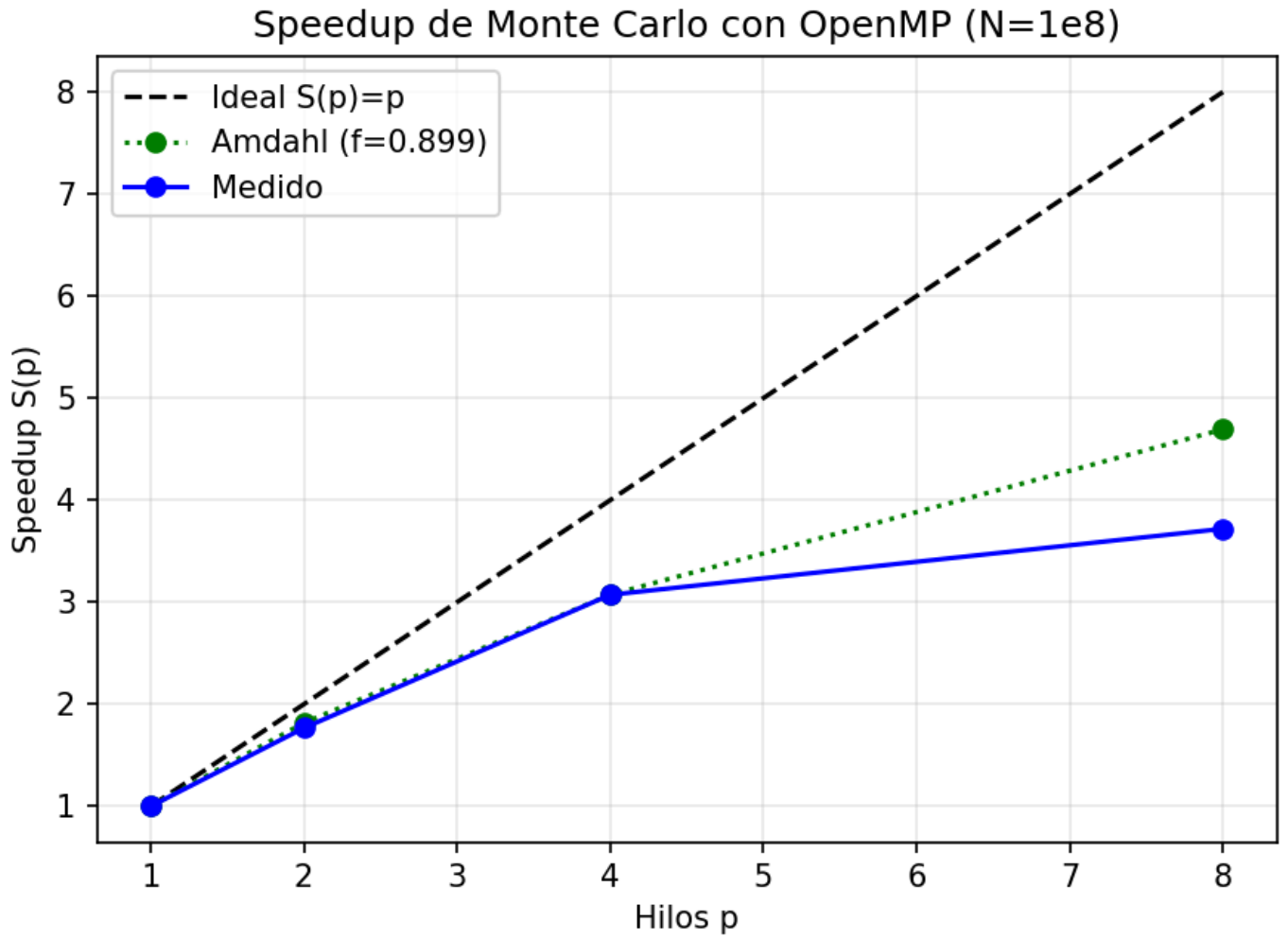
Característica	Valor
Procesador	Intel Core i5-12450HX (12.ª generación, arquitectura híbrida)
Procesadores lógicos	12 (`siblings: 12`)
Caché	12 288 KB (L3 compartida)
Frecuencia reportada	2690 MHz (nominal; la real varía con el turbo que aplica el equipo)
Extensiones	AVX, AVX2, FMA, AVX-VNNI

Característica	Valor
Estructura	`core id` = 0, 4, 8, 12 con dos procesadores lógicos cada uno, y 20, 21, 22, 23 con uno cada uno

Tabla de resultados ($N = 10^8$, mediana de 5 corridas)

p (hilos)	Mediana de T(p) [s]	S(p) = T(1)/T(p)	E(p) = S(p)/p	Karp–Flatt e
1	0,212	1,000	1,000	—
2	0,120	1,767	0,883	0,132
4	0,069	3,072	0,768	0,101
8	0,057	3,719	0,465	0,164

Tiempos individuales (s): $p = 1$: 0,214 · 0,221 · 0,195 · 0,199 · 0,212; $p = 2$: 0,099 · 0,142 · 0,137 · 0,120 · 0,115; $p = 4$: 0,074 · 0,069 · 0,067 · 0,074 · 0,069; $p = 8$: 0,056 · 0,057 · 0,057 · 0,055 · 0,063.



(Gráfica de $S(p)$ frente a p con la recta ideal, generada con `grafica.py`; el archivo es `speedup.png`.)

III.2 Estimación de la fracción paralelizable f

Se despeja f de la ley de Amdahl con el speedup medido en $p = 4$, $S(4) = 3{,}072$:

$$S = \frac{1}{(1-f)+f/p} \;\Rightarrow\; \frac{1}{S} = (1-f) + \frac{f}{p} = 1 - f\left(1 - \frac{1}{p}\right) \;\Rightarrow\; f = \frac{1-1/S}{1-1/p}$$

$$f = \frac{1-1/3{,}072}{1-1/4} = \frac{1-0{,}3255}{0{,}75} = \frac{0{,}6745}{0{,}75} \approx 0{,}899$$

Es decir, una fracción secuencial aparente $1 - f \approx 0{,}10$ (10 %).

Karp–Flatt $\left(e = \dfrac{1/S(p) - 1/p}{1 - 1/p}\right)$ mide la fracción secuencial efectiva, la que realmente se observa, y agrupa la parte secuencial verdadera con cualquier costo extra de paralelizar (creación de hilos, sincronización, desbalance, menor frecuencia etc.). Con $p = 4$ da $e = 0{,}101$, lo que coincide con el $1 - f$ anterior. La información útil está en cómo este varía, si e fuera constante, el límite sería la parte secuencial del código; como crece de 0,101 (p

$= 4$) a 0,164 ($p = 8$), parte de lo que se pierde son sobrecostos que aumentan con p, no una fracción secuencial fija. El valor de $p = 2$ ($e = 0{,}132$) rompe la tendencia, pero ese conjunto de medidas fue el más disperso (0,099 a 0,142 s, una variación del 36 %), de modo que es el menos confiable.

Dato curioso: dentro de la región cronometrada el programa casi no tiene trabajo secuencial real (solo crear el equipo de hilos y combinar p enteros). Por tanto, $f \approx 0{,}90$ no significa que el 10 % del código sea secuencial; es una fracción aparente que absorbe los sobrecostos y las limitaciones del hardware descritos en III.3.

III.3 Comparación con el ideal y con Amdahl

p	Ideal	Amdahl ($f = 0{,}899$)	Medido
2	2	1,82	1,77
4	4	3,07	3,07
8	8	4,69	3,72

- La eficiencia frente al ideal cae de 88 % ($p = 2$) a 77 % ($p = 4$) y a 46,5 % ($p = 8$). Al duplicar de 4 a 8 hilos el tiempo solo baja de 69 a 57 ms.
- Frente a Amdahl coinciden en $p = 2$ y $p = 4$ ($p = 4$ es el punto del que se calculó f), pero con $p = 8$ lo medido (3,72) queda por debajo de lo que se predijo (4,69). El modelo asume una fracción secuencial constante, y aquí hay sobrecostos que crecen con p (lo que muestra Karp–Flatt).

Factores que explican las discrepancias:

1. **Creación y destrucción de hilos, y barrera final.** El tiempo total es muy corto (57 ms con 8 hilos), de modo que unos pocos milisegundos de costo fijo pesan varios puntos porcentuales; además la resolución del reloj es de 1 ms (≈ 2 % de 57 ms).
2. **Costo de la reducción.** Es prácticamente despreciable, solo se combinan p enteros al final.
3. **Ancho de banda de memoria y jerarquía de caché.** En este programa en concreto no es un factor determinante, no hay arreglos ni acceso a memoria dentro del ciclo, todo ocurre en registros. Por eso, el límite no viene de la memoria sino de la CPU, y esto también explica que la eficiencia sea buena hasta $p = 4$.
4. **Frecuencia variable.** Con un solo hilo el procesador puede sostener su frecuencia

turbo máxima; con 4 u 8 núcleos activos, el límite de potencia y temperatura de un portátil baja la frecuencia por núcleo. Eso reduce la velocidad de cada hilo y baja el speedup respecto al ideal aun sin ninguna parte secuencial.

5. **Procesador híbrido e hiperprocesamiento.** El i5-12450HX combina núcleos P (con dos hilos por núcleo) y núcleos E, más lentos. Con `schedule(static)` cada hilo recibe exactamente N/p muestras, y el tiempo total lo fija el hilo más lento. Con $p = 8$, parte de los hilos probablemente se ejecutan en núcleos E, lo que es consistente con la mejora tan pequeña de 4 a 8 hilos. *(Esto es una hipótesis coherente con los datos, no una comprobación: la mejoría mínima de `dynamic` en II.5, 51 frente a 53 ms, apunta en la misma dirección. att: Claude)*
6. **Variabilidad de las mediciones.** Con $p = 2$ el rango de tiempos fue de 0,099 a 0,142 s, por la interferencia del sistema operativo y otros procesos; por eso se usa la mediana.

III.4 Máquina con 64 núcleos

Con $f \approx 0{,}899$:

$$S(64) = \frac{1}{0{,}101 + 0{,}899/64} = \frac{1}{0{,}1150} \approx 8{,}7, \qquad S_{\max} = \frac{1}{1-f} \approx 9{,}9.$$

Con 64 núcleos el tiempo bajaría de 0,212 s a unos 0,024 s, un speedup de apenas 8,7 y una eficiencia de solo 14 %.

Aumentar p con N fijo no mejora la precisión, solo el tiempo ya que el error sigue siendo $\approx 1{,}6 \times 10^{-4}$ porque depende solo de N (como se vio en I.3). Aumentar N mejora la precisión, pero con costo proporcional en tiempo, con $N = 4 \times 10^8$ el error baja a la mitad ($\approx 8{,}2 \times 10^{-5}$) y, en un solo hilo, el tiempo se cuadruplica. Lo útil de tener más núcleos es que permiten aumentar N sin que se alargue el tiempo, ya que en el mismo tiempo que hoy toma $N = 10^8$, 64 núcleos podrían procesar unas 8,7 veces más muestras, reduciendo el error en un factor $\sqrt{8{,}7} \approx 2{,}9$.

III.5 (Opcional) Amdahl vs. Gustafson

Amdahl fija el tamaño del problema y pregunta cuánto más rápido se resuelve. Gustafson fija el tiempo de cómputo y pregunta cuánto más grande puede ser el problema: con más procesadores, en la práctica se resuelven problemas mayores, y la parte paralelizable crece mientras la secuencial se mantiene casi constante. El speedup escalado es

$$S_G(p) = (1 - f') + f'\,p,$$

que crece linealmente con p y no tiene techo. Amdahl es pesimista porque supone que la fracción secuencial es constante cuando se escala el problema; en Monte Carlo, por ejemplo, con más núcleos se pueden usar más muestras en el mismo tiempo, lo que mejora la precisión, y la parte secuencial (lectura de argumentos, impresión) no crece.